

GStreamer Pipeline

An Introduction for Beginners

Ramakrishna Mission Residential College (Autonomous)
Narendrapur

June 16, 2026

Table of Contents

- ➊ Introduction
- ➋ What is GStreamer?
- ➌ Architecture
- ➍ Core Concepts
- ➎ Pipeline Basics
- ➏ Pipeline Design
- ➐ GStreamer Workflow
- ➑ Pipeline Examples
- ➒ Why Use GStreamer?
- ➓ Real-World Applications

- GStreamer is a pipeline-based multimedia framework that links together a wide variety of media processing systems to complete complex workflows.
- GStreamer can be used to build a system that reads files in one format, processes them, and exports them in another. The formats and processes can be changed in a plug and play fashion.
- GStreamer supports a wide variety of media-handling components, including simple audio playback, audio and video playback, recording, streaming and editing.
- The pipeline design serves as a base to create many types of multimedia applications such as video editors, transcoders, streaming media broadcasters and media players.
- GStreamer is free and open-source software. It is designed to work on a variety of operating systems, e.g. the BSDs, OpenSolaris, Linux, Android, macOS, iOS, Windows, IBM i(OS/400).

What is GStreamer?

Definition

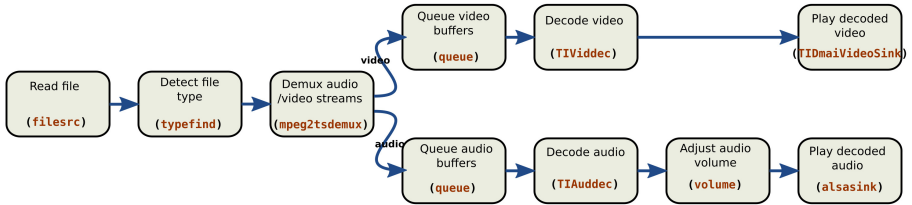
GStreamer is a **free, open-source, cross-platform multimedia framework** that enables the construction of graphs of media-handling components.

- Written in **C**, with bindings for Python, Java, and more
- Based on a **plugin architecture** — extend via plugins
- Supports MP4, MKV, MP3, H.264, VP8, FLAC, and many more formats
- Runs on Linux, Windows, macOS, Android, and iOS
- Licensed under **LGPL** — safe for commercial use

Example GStreamer Pipeline

The goals of GStreamer are to separate the application (e.g. Video player, Video editor, etc.) from the streaming media complexity (e.g. hardware acceleration, remoteness)
GStreamer – streaming media
D-Bus – inter process communication
Use **gst-launch** command to create the GStreamer pipeline

```
gst-rtsp-server - v4l2src ! video/x-raw,width=1280,height=720 ! omxh264enc ! video/x-h264,profile=baseline ! h264parse config-interval=1 ! rtph264pay name=pay0 pt=96
```



```
gst-launch filesrc location="video.ts" ! typefind ! mpeg2tsdemux name=demux \
demux. ! 'video/x-h264' ! queue ! TIIViddec ! TIDmaVideoSink \
demux. ! 'audio/mpeg' ! queue ! TIAuddec ! volume volume=5 ! alsasink
```

Broad Coverage of Multimedia Technologies

GStreamer's capabilities can be extended through new plugins. Below is a rough overview of what is available using GStreamer's own plugins.

- **Container Formats**

asf, avi, 3gp/mp4/mov, flv,
mpeg-ps/ts, mkv/webm, mxf, ogg

- **Streaming**

http, mms, rtsp

- **Codecs**

FFmpeg, various codec libraries,
3rd party codec packs

- **Metadata**

Native container formats with
a common mapping between them

- **Video**

Various colorspace, progressive
and interlaced video support

- **Audio**

Integer and float audio in various
bit depths and multichannel
configs

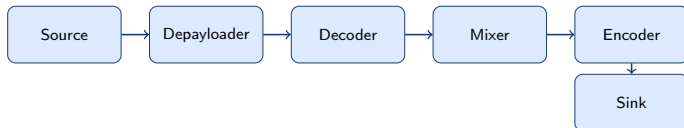
GStreamer Architecture

Core Layer

- Pipeline engine
- Element base classes
- Buffer and event system
- Clock and synchronisation

Plugin Layer

- Source plugins
- Decoder / Encoder plugins
- Filter and transform plugins
- Sink plugins



GStreamer tools

gst-launch
gst-inspect
gst-editor
gst-rtsp-server
...

media player

3rd party multimedia applications

VoIP & video
conferencing

streaming
server

video editor

GStreamer core framework

sources:
- alsa
- v4l2
- tcp/udp
- ...

sinks:
- alsa
- xvideo
- tcp/udp
- ...

containers:
- mkv
- m4v
- ogg
- ...

decoders:
- mp3
- mpeg4
- vorbis
- ...

encoders:
- daala
- h.264x
- vorbis
- ...

filters:
- converters
- mixers
- effects
- ...

protocols:
- file
- http
- rtsp
- ...

GStreamer plugins

GStreamer includes over 150 plugins
software plugins are executed on the host CPU
hw accelerated plugins shunt execution to acceleration DSPs and the result back

3rd party plugins

Texas Instruments, Motorola,
et al.

Optional additional libraries:

DMAI

imx-lib

...

D-Bus

Mesa 3D

ALSA V4L2 DRM/KMS FB IP/TCP/UDP

Linux kernel

Linux hardware device drivers

CPU + system memory

peripheral hw

acceleration hw (DSPs, etc.)

Texas Instruments DaVinci™, Ducati™
Motorola i.MX 51 bla
AMD UVD, VCE, TrueAudio™
Nvidia PureVideo,
Intel Quick Video Sync

Element

An element is the most important class of objects in GStreamer. Create a chain of elements linked together and let data flow through this chain of elements. An element has one specific function, which can be the reading of data from a file, decoding of this data or outputting this data . By chaining together several such elements like a pipeline that can do a specific task, for example media playback or capture. GStreamer ships with a large collection of elements by default.

- In the context of plugin development, an element is an object derived from the `GstElement` class. Elements provide some sort of functionality when linked with other elements: For example, a source element provides data to a stream, and a filter element acts on the data in a stream. Without elements, GStreamer is just a bunch of conceptual pipe fittings with nothing to link. A large number of elements ship with GStreamer, but extra elements can also be written. Just writing a new element is not entirely enough, however: You will need to encapsulate your element in a plugin to enable GStreamer to use it.

Element	Task
filesrc	Read data from a file
decodebin	Auto-detect and decode media
videoconvert	Convert video format
autovideosink	Display video on screen

Plugins

A plugin is essentially a loadable block of code, usually called a shared object file or a dynamically linked library. A single plugin may contain the implementation of several elements, or just a single one. For simplicity, this guide concentrates primarily on plugins containing one element.

- The plugin mechanism is used everywhere in GStreamer, even if only the standard packages are being used. A few very basic functions reside in the core library, and all others are implemented in plugins. A plugin registry is used to store the details of the plugins in a binary registry file. This way, a program using GStreamer does not have to load all plugins to determine which are needed. Plugins are only loaded when their provided elements are requested.

Inheritance Tree

GObject

└─ GInitiallyUnowned

└─ GstObject

└─ GstPlugin

GStreamer

```
graph TD; GStreamer[GStreamer] --- S(( )); S --- Source[Source plug-in]; GStreamer --- T(( )); T --- Transform[Transform plug-in]; GStreamer --- K(( )); K --- Sink[Sink plug-in];
```

Module that reads from the hard disk

Source plug-in

Module that transforms the .ogg files into raw data form, which a sound driver can understand.

Transform plug-in

Module that talks to the sound card and feeds it with data

Sink plug-in

- Pads are used to negotiate links and data flow between elements in GStreamer.
- A pad can be viewed as a “place” or “port” on an element where links may be made with other elements, and through which data can flow to or from those elements.
- Pads have specific data handling capabilities:
 - ❶ A pad can restrict the type of data that flows through it.
 - ❷ Links are only allowed between two pads when the allowed data types of the two pads are compatible.
- An analogy may be helpful here. A pad is similar to a plug or jack on a physical device. Consider, for example, a home theater system consisting of an amplifier, a DVD player, and a (silent) video projector. Linking the DVD player to the amplifier is allowed because both devices have audio jacks, and linking the projector to the DVD player is allowed because both devices have compatible video jacks. Links between the projector and the amplifier may not be made because the projector and amplifier have different types of jacks. Pads in GStreamer serve the same purpose as the jacks in the home theater system.

What is a Pipeline?

Pipeline

A special container element that holds and manages a chain of connected elements.

- Data flows: **Source** → **Processing** → **Sink**

What is a Pipeline?

Pipeline

A special container element that holds and manages a chain of connected elements.

- Data flows: **Source** → **Processing** → **Sink**
- Pipeline manages **States**: NULL → READY → PAUSED → PLAYING

What is a Pipeline?

Pipeline

A special container element that holds and manages a chain of connected elements.

- Data flows: **Source** → **Processing** → **Sink**
- Pipeline manages **States**: NULL → READY → PAUSED → PLAYING
- **Clock** — synchronises audio and video

What is a Pipeline?

Pipeline

A special container element that holds and manages a chain of connected elements.

- Data flows: **Source** → **Processing** → **Sink**
- Pipeline manages **States**: NULL → READY → PAUSED → PLAYING
- **Clock** — synchronises audio and video
- **Bus** — message passing between elements and the application

What is a Pipeline?

Pipeline

A special container element that holds and manages a chain of connected elements.

- Data flows: **Source** → **Processing** → **Sink**
- Pipeline manages **States**: NULL → READY → PAUSED → PLAYING
- **Clock** — synchronises audio and video
- **Bus** — message passing between elements and the application
- Pipelines can be **linear**, **branched**, or **merged**

Designing a GStreamer Pipeline

Steps to design a pipeline:

- 1 Identify the **input source** (file, camera, network stream)

Design Principle

Keep it modular — **one task per element**. Use decodebin / playbin for quick prototyping.

Designing a GStreamer Pipeline

Steps to design a pipeline:

- 1 Identify the **input source** (file, camera, network stream)
- 2 Choose **demuxer** / **decoder** based on format

Design Principle

Keep it modular — **one task per element**. Use decodebin / playbin for quick prototyping.

Designing a GStreamer Pipeline

Steps to design a pipeline:

- 1 Identify the **input source** (file, camera, network stream)
- 2 Choose **demuxer** / **decoder** based on format
- 3 Add **processing** / **filter** elements if needed

Design Principle

Keep it modular — **one task per element**. Use decodebin / playbin for quick prototyping.

Designing a GStreamer Pipeline

Steps to design a pipeline:

- 1 Identify the **input source** (file, camera, network stream)
- 2 Choose **demuxer** / **decoder** based on format
- 3 Add **processing** / **filter** elements if needed
- 4 Select the appropriate **sink** (screen, speaker, file, network)

Design Principle

Keep it modular — **one task per element**. Use decodebin / playbin for quick prototyping.

Designing a GStreamer Pipeline

Steps to design a pipeline:

- 1 Identify the **input source** (file, camera, network stream)
- 2 Choose **demuxer** / **decoder** based on format
- 3 Add **processing** / **filter** elements if needed
- 4 Select the appropriate **sink** (screen, speaker, file, network)
- 5 Connect elements and set pipeline to **PLAYING** state

Design Principle

Keep it modular — **one task per element**. Use decodebin / playbin for quick prototyping.

- 1 **Initialise** GStreamer (`gst_init`)

- ➊ **Initialise** GStreamer (`gst_init`)
- ➋ **Create** elements using `gst_element_factory_make`

- ➊ **Initialise** GStreamer (`gst_init`)
- ➋ **Create** elements using `gst_element_factory_make`
- ➌ **Create** a pipeline using `gst_pipeline_new`

- ➊ **Initialise** GStreamer (`gst_init`)
- ➋ **Create** elements using `gst_element_factory_make`
- ➌ **Create** a pipeline using `gst_pipeline_new`
- ➍ **Add** elements to the pipeline

- ➊ **Initialise** GStreamer (`gst_init`)
- ➋ **Create** elements using `gst_element_factory_make`
- ➌ **Create** a pipeline using `gst_pipeline_new`
- ➍ **Add** elements to the pipeline
- ➎ **Link** elements together

- ➊ **Initialise** GStreamer (`gst_init`)
- ➋ **Create** elements using `gst_element_factory_make`
- ➌ **Create** a pipeline using `gst_pipeline_new`
- ➍ **Add** elements to the pipeline
- ➎ **Link** elements together
- ➏ Set pipeline state to **PLAYING**

- ➊ **Initialise** GStreamer (`gst_init`)
- ➋ **Create** elements using `gst_element_factory_make`
- ➌ **Create** a pipeline using `gst_pipeline_new`
- ➍ **Add** elements to the pipeline
- ➎ **Link** elements together
- ➏ Set pipeline state to **PLAYING**
- ➐ **Listen** for end-of-stream or error on the bus

- ➊ **Initialise** GStreamer (`gst_init`)
- ➋ **Create** elements using `gst_element_factory_make`
- ➌ **Create** a pipeline using `gst_pipeline_new`
- ➍ **Add** elements to the pipeline
- ➎ **Link** elements together
- ➏ Set pipeline state to **PLAYING**
- ➐ **Listen** for end-of-stream or error on the bus
- ➑ Set state to **NULL** and free resources

Play a video file:

```
gst-launch-1.0 filesrc location=video.mp4 \  
! decodebin ! autovideosink
```

Play audio from a file:

```
gst-launch-1.0 filesrc location=audio.mp3 \  
! decodebin ! audioconvert ! autoaudiosink
```

Capture from webcam and display:

```
gst-launch-1.0 v4l2src device=/dev/video0 \  
! videoconvert ! autovideosink
```

Python Example — PyGObject Bindings

```
import gi
gi.require_version('Gst', '1.0')
from gi.repository import Gst, GLib

# Initialise GStreamer
Gst.init(None)

# Build pipeline from string description
pipeline = Gst.parse_launch(
    "filesrc_location=video.mp4_"
    "!_decodebin_"
    "!_autovideosink"
)

# Set to PLAYING state
pipeline.set_state(Gst.State.PLAYING)

# Run the main loop
loop = GLib.MainLoop()
loop.run()
```

Why Use GStreamer?

Advantages

- Open-source and free
- Modular plugin system
- Hardware acceleration
- Cross-platform
- Active community
- Embedded to enterprise

vs. Alternatives

- *FFmpeg* — less modular
- *DirectShow* — Windows only
- *libvlc* — higher level, less control
- GStreamer gives best **flexibility and control**

- **Video Conferencing** — WebRTC-based systems (e.g., Jitsi)

- **Video Conferencing** — WebRTC-based systems (e.g., Jitsi)
- **Embedded Systems** — NVIDIA Jetson, Raspberry Pi pipelines

Real-World Applications

- **Video Conferencing** — WebRTC-based systems (e.g., Jitsi)
- **Embedded Systems** — NVIDIA Jetson, Raspberry Pi pipelines
- **Broadcasting** — live stream encoding via RTSP / RTMP

- **Video Conferencing** — WebRTC-based systems (e.g., Jitsi)
- **Embedded Systems** — NVIDIA Jetson, Raspberry Pi pipelines
- **Broadcasting** — live stream encoding via RTSP / RTMP
- **Automotive** — in-vehicle infotainment (GENIVI platform)

Real-World Applications

- **Video Conferencing** — WebRTC-based systems (e.g., Jitsi)
- **Embedded Systems** — NVIDIA Jetson, Raspberry Pi pipelines
- **Broadcasting** — live stream encoding via RTSP / RTMP
- **Automotive** — in-vehicle infotainment (GENIVI platform)
- **IP Cameras** — RTSP stream processing and recording

Real-World Applications

- **Video Conferencing** — WebRTC-based systems (e.g., Jitsi)
- **Embedded Systems** — NVIDIA Jetson, Raspberry Pi pipelines
- **Broadcasting** — live stream encoding via RTSP / RTMP
- **Automotive** — in-vehicle infotainment (GENIVI platform)
- **IP Cameras** — RTSP stream processing and recording
- **Smart TVs** — media playback on Linux-based smart TVs

- **Video Conferencing** — WebRTC-based systems (e.g., Jitsi)
- **Embedded Systems** — NVIDIA Jetson, Raspberry Pi pipelines
- **Broadcasting** — live stream encoding via RTSP / RTMP
- **Automotive** — in-vehicle infotainment (GENIVI platform)
- **IP Cameras** — RTSP stream processing and recording
- **Smart TVs** — media playback on Linux-based smart TVs
- **GNOME Desktop** — default multimedia backend

- **Video Conferencing** — WebRTC-based systems (e.g., Jitsi)
- **Embedded Systems** — NVIDIA Jetson, Raspberry Pi pipelines
- **Broadcasting** — live stream encoding via RTSP / RTMP
- **Automotive** — in-vehicle infotainment (GENIVI platform)
- **IP Cameras** — RTSP stream processing and recording
- **Smart TVs** — media playback on Linux-based smart TVs
- **GNOME Desktop** — default multimedia backend
- **IoT Devices** — audio/video on resource-constrained hardware

Conclusion

- GStreamer is a **powerful, flexible** multimedia framework for all levels

Further Reading

<https://gstreamer.freedesktop.org/documentation/>

Conclusion

- GStreamer is a **powerful, flexible** multimedia framework for all levels
- Plugin-based architecture makes it easy to **extend and customise**

Further Reading

<https://gstreamer.freedesktop.org/documentation/>

Conclusion

- GStreamer is a **powerful, flexible** multimedia framework for all levels
- Plugin-based architecture makes it easy to **extend and customise**
- Pipelines allow modular, readable, and maintainable media processing

Further Reading

<https://gstreamer.freedesktop.org/documentation/>

Conclusion

- GStreamer is a **powerful, flexible** multimedia framework for all levels
- Plugin-based architecture makes it easy to **extend and customise**
- Pipelines allow modular, readable, and maintainable media processing
- Supports both **rapid prototyping** and **production deployment**

Further Reading

<https://gstreamer.freedesktop.org/documentation/>

Conclusion

- GStreamer is a **powerful, flexible** multimedia framework for all levels
- Plugin-based architecture makes it easy to **extend and customise**
- Pipelines allow modular, readable, and maintainable media processing
- Supports both **rapid prototyping** and **production deployment**
- Widely adopted — desktops, embedded systems, and automotive platforms

Further Reading

<https://gstreamer.freedesktop.org/documentation/>

Conclusion

- GStreamer is a **powerful, flexible** multimedia framework for all levels
- Plugin-based architecture makes it easy to **extend and customise**
- Pipelines allow modular, readable, and maintainable media processing
- Supports both **rapid prototyping** and **production deployment**
- Widely adopted — desktops, embedded systems, and automotive platforms
- An excellent choice for any multimedia project

Further Reading

<https://gstreamer.freedesktop.org/documentation/>

Thank You

Questions and Discussion

Basabraj

Ramakrishna Mission Residential College (Autonomous)
Narendrapur

<https://gstreamer.freedesktop.org>